

Week 5: Tutorial

Welcome to Week 5 tutorial. Today, you will complete the following exercises with guidance from your tutor:

- `addEventListener()`: a simple way for user control
- `setInterval()`: automate a periodic task

Set up:

This is the HTML file.

```
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta http-equiv="X-UA-Compatible" content="IE=edge" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>IDEA9103 Week 5 Tutorial</title>
  </head>
  <body>
    <svg id="basesvg" width="800" height="600"></svg>
    <script src="utils.js"></script>
    <script src="script.js"></script>
  </body>
</html>
```

Please also add the `utils.js` file from [last week's tutorial \(https://canvas.sydney.edu.au/courses/72128/pages/week-4-tutorial\)](https://canvas.sydney.edu.au/courses/72128/pages/week-4-tutorial), it will be used for all of our work today.

Let's get started!

1: `addEventListener()`: simple way for user control

Remember the 10 identical rectangles in a single row we drew last week? Let's add some simple user control to it so that a new rectangle is drawn upon each user click.

Objective:

Add user interaction to our SVG canvas by allowing them to draw a new rectangle with each click. Each new rectangle should be positioned slightly offset from the last. After 10 rectangles are added, the next click will clear the canvas.

Challenges:

- Using the desired size of our rectangles and their spacing, calculate the required size of the SVG we need to fit them in.
- Implement user interaction to add a new rectangle to the SVG canvas with each click.
- Incrementally offset the positions of each new rectangle.
- Once ten rectangles are drawn we need to remove the rectangles from the canvas.

Desired Results:

- A user can click on the SVG canvas to draw a new rectangle. Each rectangle is drawn to the right of the last, so that when all 10 are drawn they are equally spaced across the SVG.
- After the tenth rectangle, any additional click resets the canvas and starts the drawing process from scratch.

Solutions:

- We will calculate the required size of our SVG to perfectly fit 10 rectangles and 11 sections of spacing (1 before all rectangles, 1 between each rectangle and 1 after all rectangles)
- We will use the `addEventListener()` method to add user interaction. `addEventListener()` allows us to listen for user actions, such as clicks, and execute code in response.
- We will use the `replaceChildren()` method to clear the canvas. `replaceChildren()` is a function that will replace all child elements of the target element, the child elements will be replaced by whatever is in the parenthesis. If there is nothing passed into the function (nothing in the parenthesis) all the child elements of the target element will be replaced with nothing, or cleared.

Implementation:

1. Setup Initial SVG Canvas:

- Define the size of each rectangle and the spacing between them.
- Retrieve the SVG element from the HTML using `document.getElementById()`.
- Define the size of the SVG based on the rectangle size and spacing.
- Give our SVG a border so we can see the edges clearly.

2. Dynamic Rectangle Drawing:

- Create a counter to keep track of how many times the element has been clicked.
- Attach an `addEventListener` to the SVG canvas to listen for click events.

- On each click, a new rectangle is created and added to the canvas at the appropriate position, based on a counter.
- The counter is incremented with each click, determining the position of the next rectangle.

3. Canvas Reset after Ten Rectangles:

- Check if ten rectangles have been added by comparing the size of the counter.
- If the counter is higher than 10, clear all the child elements and reset the counter to zero, ready to start again.

```
//Get the svg element from the HTML
let svg = document.getElementById("basesvg");

let rectWidth = 50;
let rectHeight = 50;
let rectSpacing = 20;
let rectYPos = 20;
let counter = 0;

//lets resize the svg to fit the rectangles
svg.setAttribute("width", rectSpacing + (10 * (rectWidth + rectSpacing)));
svg.setAttribute("height", rectYPos + rectHeight + rectSpacing);
//lets give the svg a border so we can see it
svg.setAttribute("style", "border: 1px solid black");
/*
Instead of using a for-loop, we use addEventListener to capture user click
and trigger the drawing of a new rectangle

We attach the event listener to the whole svg element, and we are listening for a "click" even
t
The function that we pass to the event listener will be called when the event is triggered,
in this case the function is defined inside the event listener method.
*/
svg.addEventListener("click", function(e) {
    let newRect = createRect(rectSpacing + (counter * (rectWidth + rectSpacing)), rectYPos, re
ctWidth, rectHeight, null, null, null);
    svg.appendChild(newRect);
    counter++;
    //if we have added 10 rectangles, we will clear the svg and reset the counter
    if(counter > 10){
        counter = 0;
        //This clears the svg by replacing all the children with nothing, but it can replace t
he
        //children with other elements if you had them prepared
        svg.replaceChildren();
    }
});
```

2: setInterval(): automate a periodic task

What if we want the rectangles to appear one by one automatically, following a fixed rhythm?

Objective:

Instead of the user clicking on the SVG to add a rectangle, add and eventually clear the rectangles automatically.

Challenges:

- Use a timer to automate the drawing process at fixed intervals.

Desired Results:

- Rectangles appear on the canvas one by one with a set delay, creating an automated drawing effect.
- After the tenth rectangle is drawn, the canvas automatically clears itself, and starts the sequence over again.

Solutions:

- We will move our drawing and clearing code from the previous task into a function of its own, without the `addEventListener()`.
- We will use `setInterval()` to periodically invoke the function. `setInterval()` is a function that takes two parameters, a function to run and an interval (how many seconds between running the function again). This interval is set to trigger every second, but can be adjusted as needed, and in our code the interval will run indefinitely.

Implementation:

We will use most of the code from the previous task.

Create the stand-alone function:

- Using the code from the previous task, remove the event listener and create a stand-alone function called `drawNextRect()`.

Automated Rectangle Drawing:

- Employ `setInterval()` to execute the `drawNextRect()` function at regular intervals, specified in milliseconds.

```
//Get the SVG element from the HTML
let svg = document.getElementById("basesvg");

let rectWidth = 50;
let rectHeight = 50;
let rectSpacing = 20;
let rectYPos = 20;
let counter = 0;

//Let's resize the SVG to fit the rectangles
svg.setAttribute("width", rectSpacing + (10 * (rectWidth + rectSpacing)));
svg.setAttribute("height", rectYPos + rectHeight + rectSpacing);
//lets give the svg a border so we can see it
svg.setAttribute("style", "border: 1px solid black");

/*
Instead of the event listener, we will define a function that will add each rectangle
to the SVG, and reset the counter when it reaches 10, and clear the svg.
*/
```

```
function drawNextRect() {
  let newRect = createRect(rectSpacing + (counter * (rectWidth + rectSpacing)), rectYPos, re
ctWidth, rectHeight, null, null, null);
  svg.appendChild(newRect);
  counter++;
  //if we have added 10 rectangles, we will clear the SVG and reset the counter
  if(counter > 10){
    counter = 0;
    //This clears the SVG by replacing all the children with nothing, but it can replace t
he
    //children with other elements if you had them prepared
    svg.replaceChildren();
  }
}

//This is our interval in Milliseconds: 1 second = 1000 Milliseconds
let stepInterval = 1000;

//The setInterval function takes two arguments, the function to run, and the interval in milli
seconds to run it
//This means that the drawNextRect function will run every 1 second. It will run forever
setInterval(drawNextRect, stepInterval);
```

Looks like the rectangles are a bit more animated now, right?

3: Arcs and paths

Paths are another kind of SVG object; they can be used to draw complex shapes, including curves. When we draw a curve in a path, it is always considered as a section of an ellipse, so we describe how much of the ellipse we want to draw, the size of the circle we are drawing a section of, the start point and end point and the rotation. Because SVG paths are so flexible they have a complex system for controlling what is drawn and how, and use a large set of parameters.

For the adventurous of you, head over to the MDN docs to see a full description of paths and how they work <https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Paths>  <https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Paths>.

For now we are going to use paths to draw a simple arc.

Objective:

Our goal is to draw a quarter of a circle (or a 90-degree arc) using SVG paths. This exercise will introduce you to the basics of SVG path syntax, specifically how to create arcs.

Challenges:

- Understanding SVG path syntax, especially the commands for moving to a point ("M") and drawing an arc ("A").

Desired Results:

- A static SVG drawing that visually represents a quarter of a circle, demonstrating the use of paths to create curved shapes.
- The arc should start at the 3 o'clock position and end at the 6 o'clock position on a hypothetical clock face, forming a right angle or a quarter-circle.

Solutions:

- We will define a length and start point all in radians for our arc.
- We will use those to calculate the end point in radians.
- We will use the start and end points in radians to calculate the start and end coordinates of the arc.
- We will create an SVG path.
- We will use the SVG Path Commands to create the arc. We will set the data attribute to the command for an arc using the following:
 - The "M" command (move), sets the starting point
 - The "A" command defines the type of path as an arc
 - The parameters for the "A" command include the x and y radii of the arc (equal for a circle), flags to determine the arc sweep,
 - Finally we specify the coordinates of the arc's end point.

```
//Get the svg element from the HTML
let svg = document.getElementById("basesvg");

let circleXPos = 200;
let circleYPos = 200;
let radius = 100;

//This defines how much of the circle we want to draw in radians
//In this case it is Math.PI / 2 radians, or 90 degrees, or 1/4 of the circle
let segmentRadians = Math.PI / 2;

//The segment starts from angle 0, as we know this is 3 o'clock on the circle
let startAngle = 0;

//The end angle is just the start angle plus the segmentRadians
//As this is Math.PI / 2 radians, or 90 degrees, the end will be 6 o'clock
let endAngle = segmentRadians;

//Calculate the start and end positions of the path (segment) using the start and end angles
let startXPos = circleXPos + radius * Math.cos(startAngle);
let startYPos = circleYPos + radius * Math.sin(startAngle);
let endXPos = circleXPos + radius * Math.cos(endAngle);
let endYPos = circleYPos + radius * Math.sin(endAngle);

//Draw the path that represents the segment
//First we need a path element
let segment = document.createElementNS("http://www.w3.org/2000/svg", "path");
/*
  Lets look in detail at how we can use the path type to draw segments of a circle

  The "d" attribute of the path is the path data, everything after the comma is what this will be set to
```

The "M" means move to, this is where we start to draw the path (the x and y position)
 The "A" means Arc to, so we are drawing a curve
 The curve could be from an ellipse so it needs a radius for the x and y axis,
 in our case they are the same because we are drawing a regular circle
 The first 0 means we are NOT rotating the circle,
 The next 0 means a normal arc size, not a large arc size
 The 1 means we are drawing the circle in a positive (clockwise) direction
 The last two numbers are the end position of the arc
 We use the backtick notation to put our variables in the string that defines the path
 */

```
segment.setAttribute("d", `M ${startXPos} ${startYPos} A ${radius} ${radius} 0 0 1 ${endXPos}
${endYPos}`);
segment.setAttribute("stroke", "black");
segment.setAttribute("stroke-width", 2);
segment.setAttribute("fill", "none");

svg.appendChild(segment);
```

4: An interactive clock

With the knowledge we learned above, let's dive into creating an interactive clock - every time our user clicks the clock, its hand will point to the next number!

Objective:

Develop an interactive SVG clock where the hour hand advances to the next hour upon each user click.

Challenges:

- Calculating the correct positions for the clock numbers around the circle, ensuring they are evenly spaced.
- Drawing the hour hand in such a way that it points accurately to the numbers around the clock, adjusting its angle with each click.

Desired Results:

- A fully functional SVG clock displayed on the webpage, with numbers 1 through 12 evenly distributed around the clock face.
- The hour hand of the clock should move to point directly at each hour number in sequence, advancing one hour with each click.
- After reaching 12, the next click should bring the hour hand back to 1, demonstrating a complete cycle.

Solutions:

- We will calculate the starting angle offset. When we use a 0 radians angle to calculate x and y positions on a circle, it points to 3 o'clock, and we want to start at 12 so we need to calculate the offset in radians.

- We will create a `drawText()` function to draw text at a specific position.
- We will create a `drawClockNumbers()` function that uses the `drawText()` function to draw the clock numbers at the correct position using `Math.cos()` and `Math.sin()` to calculate the x and y positions for each clock number based on their angle in radians. This involves using the circle's centre as the origin and placing each number at a specific distance from the centre, using a slightly larger radius than the circle of the clock face.
- We will create a `drawHourHand()` function to draw the hour hand at a specific position. The hour hand will be a line with the start at the centre of the circle and the end will point at the correct number for the time. The end position will be calculated using `Math.cos()` and `Math.sin()`.
- We will create a `drawClock()` function that will draw the circle of the clock face and then call the `drawClockNumbers()` and the `drawHourHand()` functions to draw the whole clock.
- We will add an event listener to the SVG with an inline function that increments the counter that keeps track of the time and calls the `drawClock()` function.

Implementation:

Here's a summary of how each part of the interactive clock is implemented:

1. **Set Up the SVG Canvas:** Define the dimensions of the SVG element to ensure there is enough space for the clock and its components.
2. **Draw the Clock Numbers:** For each number 1 through 12, calculate its position on the clock face using its angle in radians. The angle is adjusted based on the number's sequence around the clock.
3. **Animate the Hour Hand:** On each click, increase the counter and calculate the new position of the hour hand based on this counter. The hour hand's angle is adjusted to point at the correct hour number.
4. **Interactive Click Event:** Attach a click event listener to the SVG element to increment the hour counter and redraw the clock face and hand accordingly.

```
let svg = document.getElementById("basesvg");

let width = 400;
let height = 400;

svg.setAttribute("width", width);
svg.setAttribute("height", height);

// This function draws some text at position x, y. This function can be added to the utils.js file.
function drawText(x, y, text) {
  let newText = document.createElementNS("http://www.w3.org/2000/svg", "text");
  newText.setAttribute("x", x);
```



```

    newText.setAttribute("y", y);
    newText.innerHTML = text;
    svg.appendChild(newText);
}

let clockX = width/2;
let clockY = height/2;
let clockSize = width/3;

/*
  We want draw our hand and numbers around a full circle
  and since JavaScript only has a constant for a
  half-circle (Math.PI) we should create a constant for two PI
  as we'll be using it for a few calculations
*/
const TWO_PI = 2*Math.PI;

/*
  Multiplying a radius to an initial polar angle of 0 PI will
  shoot off into the right toward 3 o'clock
  Since we want our initial direction or ray to go toward the top
  at 12 o'clock, we need to provide an offset when we calculate our
  polar angle
*/
let angleOffset = -Math.PI/2

/*
  counter is used to advance the polar angle which determines the
  direction of the hour hand
  It's incremented by 1 with each mouse click
*/
let counter = 0;

/*
  This function draws a line from the clock's position to a polar
  coordinate
  We already know the first pair of coordinates clockX and clockY
  so we just need to calculate the last pair of coordinates
*/
function drawHourHand() {
  // First calculate the angle that determines the direction of the hour hand
  let angle = angleOffset + TWO_PI/12 * counter;

  /*
    Then feed the angle into the Math.cos()/Math.sin() functions and
    multiply by the radius
  */
  let hourHandLength = clockSize/2;
  let x2 = clockX + Math.cos(angle) * hourHandLength;
  let y2 = clockY + Math.sin(angle) * hourHandLength;
  /*
    clockX and clockY are added otherwise the coordinates
    will circle around the top-left corner of the element
  */

  let newLine = createLine(clockX, clockY, x2, y2, 2, 255, 255, 255);
  svg.appendChild(newLine);
}

// This function draws the numbers around the clock
function drawClockNumbers() {
  /*
    A for-loop is used here since we can use the iterator
    i to advance the angle used to calculate the direction
    in which the number is drawn
    i is initialised at 1 and the condition for the loop
    to continue checks that i <= 12 so that we can use i
    for drawText()'s third parameter
  */
  for (let i=1; i<=12; i++) {
    let angle = angleOffset + TWO_PI/12 * i;
    let numberRadius = clockSize*0.9;

```

```

    let x = clockX + Math.cos(angle) * numberRadius;
    let y = clockY + Math.sin(angle) * numberRadius;

    drawText(x, y, i);
  }
}

// This function brings together the functions we've defined to draw
// all the features of our clock
function drawClock() {
  // draw the clock base circle
  let newCircle = createCircle(clockX, clockY, clockSize, 127, 127, 127);
  svg.replaceChildren();
  svg.appendChild(newCircle);

  drawClockNumbers();

  drawHourHand();
}

// Draw the initial clock
drawClock();

// Add a click event listener to the element
svg.addEventListener("click", function() {
  counter++;
  drawClock();
})

```

With the exercises done earlier, do you know how to make the clock hand rotate automatically using `setInterval()`?

Try to use the code below to replace the event listener:

```

let stepInterval = 1000;

function autoClock() {
  counter++;
  drawClock();
}
setInterval(autoClock, stepInterval);

```

Inspiration for the week

This week's inspiration is complex and amazing. We want to show you a work from Japanese artist Ryoji Ikeda () called data-verse. This work is composed of relatively simple images and sounds, but is orchestrated and presented with great artistic care and inspiration. His work is known for its meticulous precision and complexity, combining large-scale video projections with minimalist electronic soundtracks to create immersive experiences.

His work "data-verse" premiered at the Venice Biennale in 2019. You can find a great video of it here:

<https://www.youtube.com/watch?v=S-vSFDZGfF4>  (<https://www.youtube.com/watch?v=S-vSFDZGfF4>)



<https://www.youtube.com/watch?v=S-vSFDZGfF4>

And some more information about the work here:

<https://www.audemarspiguat.com/com/en/news/art/ryoji-ikeda-data-verse-trilogy.html> 
(<https://www.audemarspiguat.com/com/en/news/art/ryoji-ikeda-data-verse-trilogy.html>)

And his website containing some information about his many other works is here:

<https://www.ryojiikeda.com/>  (<https://www.ryojiikeda.com/>)

Ryoji Ikeda is an incredibly talented artist that manages to use simple ingredients to make amazing immersive results. While you are passing time on the internet, take a look at more of his work.

Happy coding!

Copyright © The University of Sydney. Unless otherwise indicated, 3rd party material has been reproduced and communicated to you by or on behalf of the University of Sydney in accordance with section 113P of the Copyright Act 1968 (Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.

Live streamed classes in this unit may be recorded to enable students to review the content. If you have concerns about this, please visit our [student guide \(https://canvas.sydney.edu.au/courses/4901/pages/zoom\)](https://canvas.sydney.edu.au/courses/4901/pages/zoom) and contact the unit coordinator.
[Privacy Statement \(https://www.sydney.edu.au/privacy-statement.html\)](https://www.sydney.edu.au/privacy-statement.html)